Memoria de Prácticas

Introducción al Aprendizaje Automático

Jesús Fernández López
Rafael Carlos Schoettel Vilchez

Abril, 2026

Capítulo 1

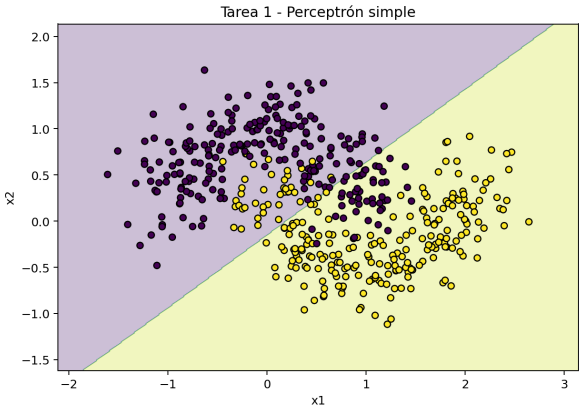
Arquitectura de Redes Neuronales: El Perceptrón Multicapa

En esta práctica nos proponemos explorar la çaja negra"de las Redes Neuronales a través del Perceptrón Multicapa (MLP). No buscamos simplemente entrenar modelos, sino entender la lógica matemática que permite a una red aprender fronteras de decisión complejas. Analizaremos cómo decisiones que parecen menores —como el número de neuronas o la función de activación— determinan radicalmente si un modelo será capaz de generalizar o si quedará atrapado en soluciones locales ineficientes.

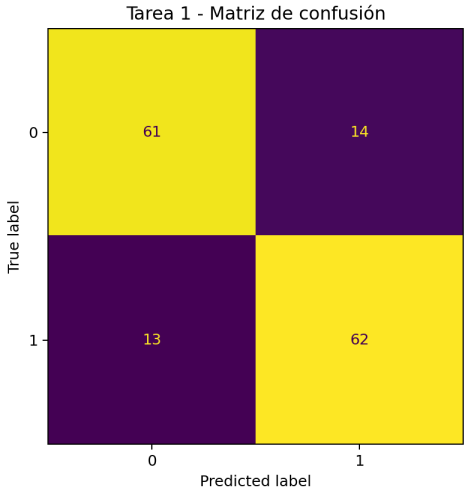
1.1. La barrera de la linealidad: El Perceptrón Simple

Para poner a prueba la red, empezamos con un clásico: el dataset `make_moons`. Con 500 muestras y un nivel de ruido generoso (25 %), la geometría resultante es un reto para cualquier clasificador lineal. Lo primero que hicimos fue probar con un MLP sin capas ocultas. En la práctica, esto es como tener un modelo logístico básico; una sola neurona intentando poner orden en un lío de puntos entrelazados.

1.1.1. Resultados y Limitación Estructural



(a) Frontera de decisión lineal.



(b) Matriz de confusión.

Figura 1.1: Rendimiento del perceptrón simple ante un problema no lineal.

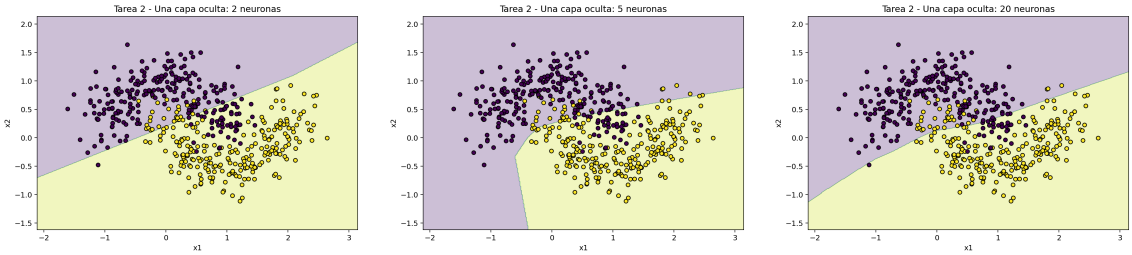
Los resultados de la Figura 1.1a son el vivo retrato de la frustración lineal: una línea recta intentando separar lo que es intrínsecamente curvo. El 86 % de acierto parece decente, pero es un techo de cristal. No importa cuánto tiempo estemos entrenando o qué tasa de aprendizaje usemos; una sola neurona no puede doblar el espacio. Esta limitación geométrica es la que nos obliga a dar el salto a arquitecturas más profundas.

1.2. Inyección de capacidad: El papel de las neuronas ocultas

La solución al problema anterior pasa por introducir una capa oculta que proyecte los datos a un espacio donde sí sean separables. En este experimento, analizamos cómo el número de neuronas ocultas (N_h) actúa como un regulador de la flexibilidad del modelo.

1.2.1. Análisis de la frontera de decisión

Hemos comparado tres configuraciones: $N_h \in \{2, 5, 20\}$.



(a) 2 neuronas: Underfitting. (b) 5 neuronas: Transición. (c) 20 neuronas: Ajuste óptimo.

Figura 1.2: Efecto de la densidad de la capa oculta en la morfología de la frontera.

Tabla 1.1: Métricas de convergencia y rendimiento según la arquitectura.

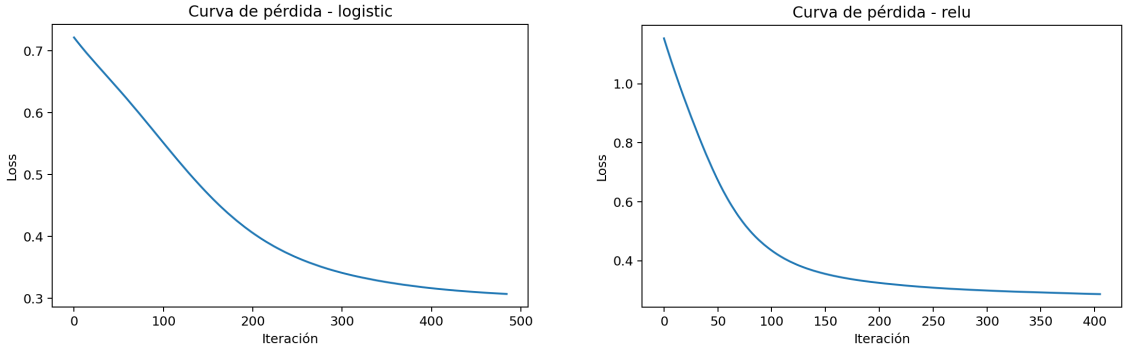
Arquitectura	**Acc. Test**	**Iteraciones**	**Observación**
MLP(2)	0.8200	648	Capacidad insuficiente
MLP(5)	0.8400	726	Esfuerzo de optimización máximo
MLP(20)	0.8533	406	Convergencia eficiente

1.2.1.0.1. Interpretación de los resultados. Se observa un fenómeno interesante: con 2 neuronas, la red es todavía demasiado rígida. Sin embargo, al llegar a 20 neuronas, la frontera logra capturar la esencia de las "moonsçon suavidad. Es notable que la red con 20 neuronas converge mucho más rápido (406 iteraciones vs. 726). Esto sugiere que, al dotar a la red de más parámetros, el optimizador (Adam) encuentra caminos más simples hacia el mínimo global, evitando quedar atrapado en valles de pérdida poco profundos que suelen aparecer en redes muy pequeñas y .estresadas".

1.3. Dinámica del aprendizaje: ReLU vs. Sigmoide

Fijando la arquitectura en 20 neuronas, comparamos la función logística tradicional (sigmoide) frente a la Rectified Linear Unit (ReLU).

1.3.1. Convergencia y Gradientes



(a) Pérdida con Sigmoide.

(b) Pérdida con ReLU.

Figura 1.3: Comparativa de la velocidad de aprendizaje.

Aunque ambas alcanzan precisiones similares (ver Tabla 1.2), ReLU es claramente superior en términos de eficiencia. La sigmoide sufre el problema de la **saturación**: cuando los valores de entrada son altos o bajos, la derivada es casi nula, lo que çongela"la actualización de los pesos. ReLU, al tener una derivada constante de 1 para todos los valores positivos, mantiene vivo el flujo del gradiente, permitiendo una convergencia un 16 % más rápida y una pérdida final más baja (0.2870).

Tabla 1.2: Comparativa de funciones de activación (Arquitectura fija: 20 neuronas).

Activación	**Acc. Test**	**Convergencia**	**Loss final**
Logistic (Sigmoide)	0.8400	485 iter.	0.3071
ReLU	0.8533	406 iter.	0.2870

1.4. El reto de los dígitos: Hacia la parsimonia estructural

El dataset de dígitos (8×8 píxeles, 10 clases) representa un salto en complejidad. El reto consiste en superar el 95 % de acierto con la arquitectura más eficiente posible.

1.4.1. Búsqueda de la arquitectura óptima

Probamos diversas configuraciones, desde redes monocapa hasta estructuras profundas.

Tabla 1.3: Evaluación de arquitecturas para el reconocimiento de dígitos.

Arquitectura	**Parámetros aprox.**	**Acc. Test**	**Iteraciones**	**Meta**
MLP(20)	1.500	0.9644	276	✓
MLP(50)	3.800	0.9711	180	✓
MLP(100)	7.500	0.9800	147	✓
MLP(50,20)	4.500	0.9756	131	✓

1.4.2. Justificación de la elección: MLP(20)

Llegados a este punto, nos planteamos: ¿necesitamos realmente cientos de neuronas? Aplicando el **principio de parsimonia**, la respuesta es un rotundo no. La arquitectura MLP(20) es nuestra gran ganadora. Logra un 96.44 % de acierto sin despeinarse y con una fracción de la carga computacional de sus hermanas mayores. En ingeniería, y más en IA, un modelo que hace el trabajo con un 80 % menos de parámetros es siempre la elección inteligente, especialmente si pensamos en dispositivos limitados o eficiencia energética.

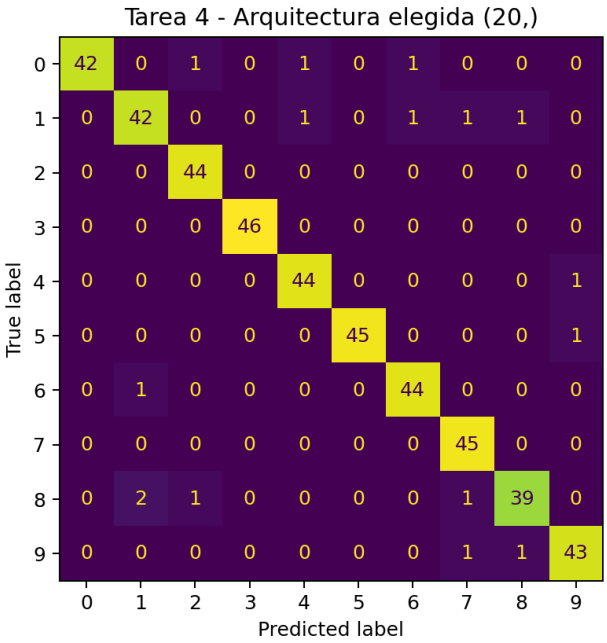


Figura 1.4: Matriz de confusión de la arquitectura propuesta MLP(20). El reconocimiento es casi impecable, con una tasa de error residual en dígitos con trazos ambiguos.

1.5. Reflexiones Finales

Tras este estudio, las conclusiones son claras: la ïnteligencia"de la red no depende exclusivamente de su tamaño, sino de la armonía entre su arquitectura y la naturaleza de los datos. Hemos aprendido que:

- El **MLP sin capas ocultas** es insuficiente para la mayoría de problemas reales debido a su naturaleza lineal.
- La **profundidad y el ancho** deben equilibrarse; a menudo, una capa pequeña bien entrenada es superior a una red profunda innecesaria.
- La función **ReLU** es el estándar de oro por su capacidad para evitar la desvanecimiento del gradiente, permitiendo entrenamientos más ágiles.
- En ingeniería, **menos es más**: la búsqueda del modelo más simple que cumpla el requisito (parsimonia) es la mejor defensa contra el sobreajuste.